



Southeast University

Introduction to Software Engineering

Wang Lulu, Liao Li

lliao@seu.edu.cn

Outline

Part 0. Overview

Part 1. Software Processes

Part 2. Modeling

Part 3. Quality Assurance

Part 4. Software Project Management

- ❖ **Chap 22. Project Management Concepts**
- ❖ **Chap 23. Process and Project Metrics**
- ❖ **Chap 24-25. Project Estimation & Scheduling**
- ❖ **Chap 26. Risk Analysis**

Chap 23. Process and Project Metrics

❖ **Why Do We Measure?**

❖ **Process Measurement and Metrics**

❖ **Project Metrics**

❖ **Typical Types of Metrics**

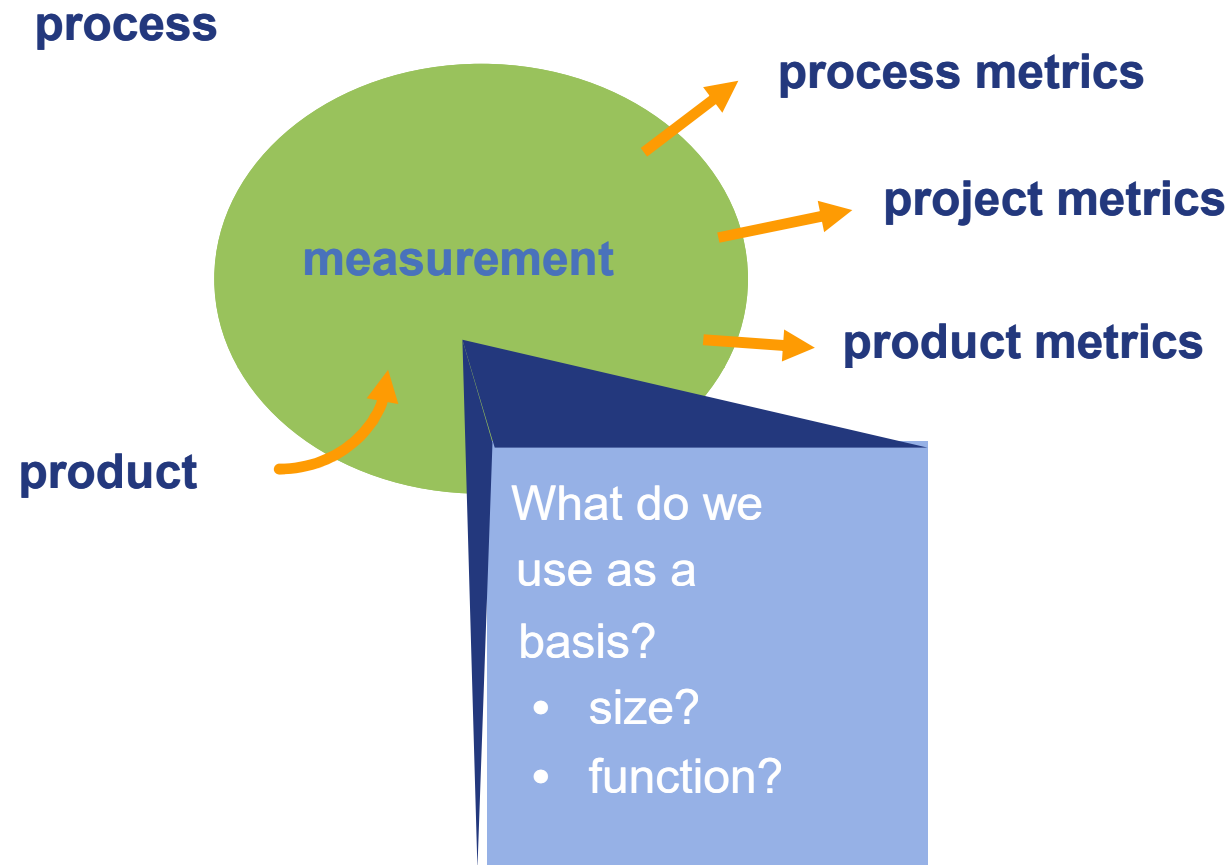
❖ **Metrics Guidelines**

❖ **Measuring Quality**

} 度量策略

测量技术

A Good Manager Measures



Why Do We Measure?

1. 度量的意义

- ❖ **Assess** the status of an ongoing project
- ❖ **Track** potential risks
- ❖ **Uncover problem areas** before they go “critical”
- ❖ **Adjust** work flow or tasks,
- ❖ **Evaluate the project team’s ability** to control quality of software work products.

1. 评估正在进行的项目的状态

2. 跟踪潜在风险

3. 在问题造成不良影响前发现风险

4. 调整工作流程或任务

5. 评估项目团队控制软件工作产品质量的能力

Process Measurement

❖ We measure the efficacy of a software process indirectly.

- 我们间接衡量软件过程的有效性。

■ That is, we derive a set of metrics based on the outcomes that can be derived from the process.

- 也就是说，我们基于过程可以得出的结果推导出一组度量标准。

■ Outcomes include

- **measures of errors uncovered before release of the software**
- **defects delivered to and reported by end-users**
- **work products delivered (productivity)**
- **human effort expended**
- **calendar time expended**
- **schedule conformance**
- **other measures.**

• 结果包括：

- 在软件发布前发现的错误的度量
- 由最终用户交付和报告的缺陷
- 交付的工作产品（生产力）
- 花费的人力
- 花费的日历时间
- 进度符合度
- 其他度量。

❖ We also derive process metrics by measuring the characteristics of specific software engineering tasks.

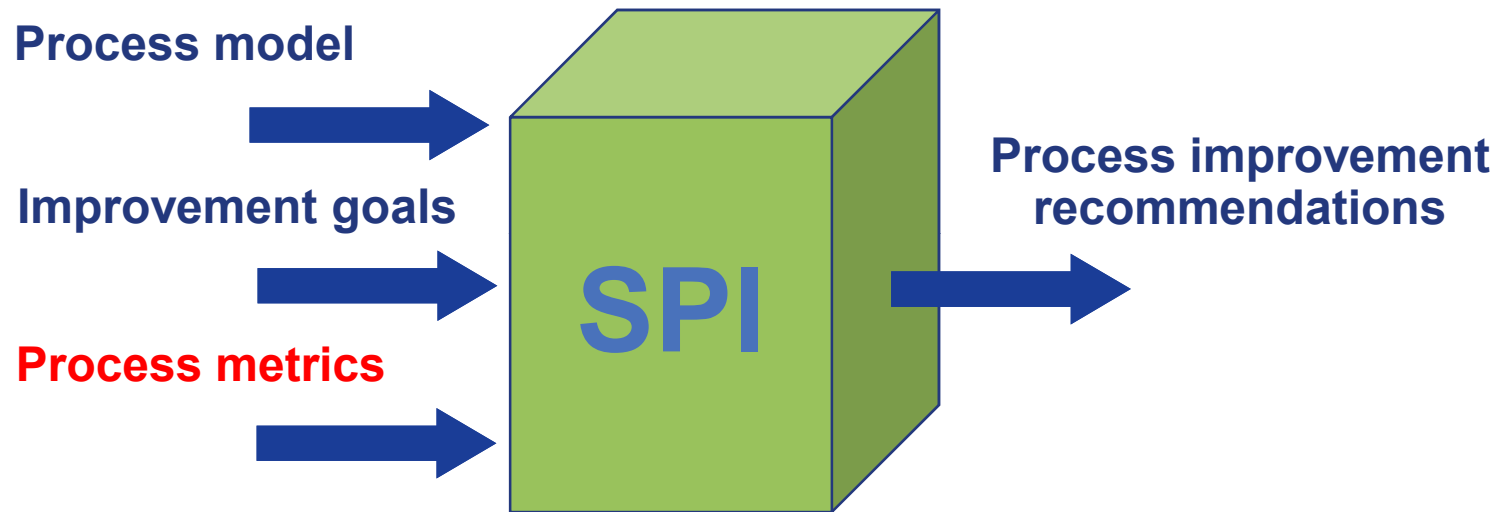
- 我们还通过测量特定软件工程任务的特征来推导过程度量标准。

2. 过程度量——间接测量软件过程的效率

Chap 23. 6/28

根据过程中获得的一系列数据或软件工程任务的特性来进行测量。

Software Process Improvement



Process Metrics

4. 过程度量的五个方面

❖ **Quality-related**

- focus on quality of work products and deliverables

❖ **Productivity-related**

- Production of work-products related to effort expended

❖ **Statistical SQA data**

- error categorization & analysis

❖ **Defect removal efficiency**

- propagation of errors from process activity to activity

❖ **Reuse data**

- The number of components produced and their degree of reusability
 - 质量相关——关注工作产品和可交付成果的质量
 - 生产力相关——工作产品的生产与付出的努力相关
 - 统计 SQA 数据——错误分类和分析
 - 缺陷去除效率——传播从流程活动到活动的错误
 - 重用数据——生产的组件数量及其可重用程度

Project Metrics

- ❖ Used to **minimize the development schedule** by making the adjustments necessary to avoid delays and mitigate potential problems and risks
- ❖ Used to **assess product quality** on an ongoing basis and, when necessary, modify the technical approach to improve quality.
- ❖ Every project should measure:
 - *inputs*—measures of the resources (e.g., people, tools) required to do the work.
 - *outputs*—measures of the deliverables or work products created during the software engineering process.
 - *results*—measures that indicate the effectiveness of the deliverables.

- 用于通过进行必要的调整来避免延误并减轻潜在问题和风险，从而最小化开发进度。
 - 用于持续评估产品质量，并在必要时修改技术方法以提高质量。
- 每个项目都应测量：
 - **输入** —— 完成工作所需资源（例如人员、工具）的度量。
 - **输出** —— 软件工程过程中创建的交付物或工作产品的度量。
 - **结果** —— 指示交付物有效性的度量。

Typical Size-Oriented Metrics

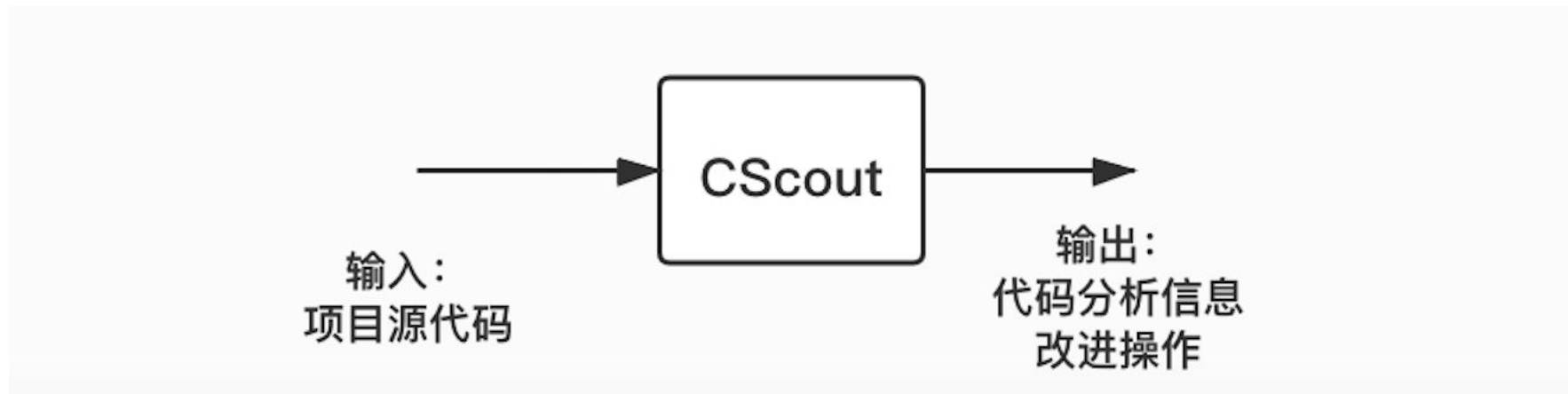
- ❖ errors per KLOC (thousand lines of code)
- ❖ defects per KLOC
- ❖ \$ per LOC
- ❖ pages of documentation per KLOC
- ❖ errors per person-month
- ❖ errors per review hour
- ❖ LOC per person-month
- ❖ \$ per page of documentation
- ❖

典型的面向大小的度量

- 每千行代码 (KLOC) 的错误数
- 每千行代码 (KLOC) 的缺陷数
- 每行代码的成本 (\$ per LOC)
- 每千行代码 (KLOC) 的文档页数
- 每人月的错误数
- 每审查小时的错误数
- 每人月的代码行数 (LOC per person-month)
- 每页文档的成本 (\$ per page of documentation)

Example

❖ C语言重构与代码分析图形工具：CScout



CScout Main Page

localhost:8081

Files

文件级别信息

- File metrics
- Browse file tree
- All files
- Read-only files
- Writable files
- Files containing unused project-scoped writable identifiers
- Files containing unused file-scoped writable identifiers
- Writable .c files without any statements
- Writable files containing unprocessed lines
- Writable files containing strings
- Writable .h files with #include directives
- Specify new file query

Identifiers

变量级别信息

- Identifier metrics
- All identifiers
- Read-only identifiers
- Writable identifiers
- File-spanning writable identifiers
- Unused project-scoped writable identifiers
- Unused file-scoped writable identifiers
- Unused writable macros
- Writable variable identifiers that should be made static
- Writable function identifiers that should be made static
- Specify new identifier query

File Dependencies

- File include graph: [writable files](#), [all files](#)
- Compile-time dependency graph: [writable files](#), [all files](#)
- Control dependency graph (through function calls): [writable files](#), [all files](#)
- Data dependency graph (through global variables): [writable files](#), [all files](#)

Functions and Macros

函数级别信息

- Function metrics
- All functions
- Project-scoped writable functions
- File-scoped writable functions
- Writable functions that are not directly called
- Writable functions that are called exactly once
- Non-static function call graph
- Function and macro call graph
- Specify new function query

Operations

简单的重构操作

- Global options — [save global options](#)
- Identifier replacements
- Function argument refactorings
- Select active project
- About CScout
- Save changes and continue
- Exit — [saving changes](#)
- Exit — [ignore changes](#)

File Metrics

Writable Files

Number of elements: 13

Metric	Total	Min	Max	Avg
Number of characters	157271	1923	43298	12097.8
Number of comment characters	30152	0	6307	2319.38
Number of space characters	28707	298	8735	2208.23
Number of line comments	0	0	0	0
Number of block comments	760	0	190	58.4615
Number of lines	6432	100	1913	494.769
Maximum number of characters in a line	1054	24	107	81.0769
Number of character strings	684	0	154	52.6154
Number of unprocessed lines	12	0	8	0.923077
Number of C preprocessor directives	274	0	92	21.0769
Number of processed C preprocessor conditionals (ifdef, if, elif)	6	0	3	0.461538
Number of defined C preprocessor function-like macros	30	0	22	2.30769
Number of defined C preprocessor object-like macros	161	0	92	12.3846
Number of preprocessed tokens	39529	393	12189	3040.69
Number of compiled tokens	44120	26	14020	3393.85
The Wireshark Network Analyzer	13	1	1	1
Number of statements	4293	0	1589	330.231
Number of defined project-scope functions	168	0	51	12.9231
Number of defined file-scope (static) functions	2	0	1	0.153846
Number of defined project-scope variables	149	1	36	11.4615
Number of defined file-scope (static) variables	109	0	92	8.38462
Number of complete aggregate (struct/union) declarations	12	0	6	0.923077
Number of declared aggregate (struct/union) members	56	0	32	4.30769
Number of complete enumeration declarations	0	0	0	0
Number of declared enumeration elements	0	0	0	0
Number of directly included files	62	0	9	4.76923

Read-only Files

Number of elements: 15

Metric	Total	Min	Max	Avg
Number of characters	41491	163	9551	2766.07
Number of comment characters	25961	54	5558	1730.73
Number of space characters	2595	13	760	173
Number of line comments	12	0	12	0.8
Number of block comments	128	0	60	8.53333
Number of lines	1188	13	274	79.2
Maximum number of characters in a line	1050	38	84	70
Number of character strings	63	0	59	4.2
Number of unprocessed lines	17	0	5	1.13333
Number of C preprocessor directives	289	1	103	19.2667
Number of processed C preprocessor conditionals (ifdef, if, elif)	27	0	6	1.8
Number of defined C preprocessor function-like macros	30	0	13	2
Number of defined C preprocessor object-like macros	87	0	31	5.8
Number of preprocessed tokens	3465	16	1068	231
Number of compiled tokens	1779	0	602	118.6
Number of copies of the file	15	1	1	1
The Wireshark Network Analyzer	0	0	0	0
Number of defined project-scope functions	0	0	0	0
Number of defined file-scope (static) functions	0	0	0	0
Number of defined project-scope variables	6	0	3	0.4
Number of defined file-scope (static) variables	0	0	0	0
Number of complete aggregate (struct/union) declarations	7	0	3	0.466667
Number of declared aggregate (struct/union) members	56	0	23	3.73333
Number of complete enumeration declarations	0	0	0	0
Number of declared enumeration elements	0	0	0	0
Number of directly included files	24	0	20	1.6

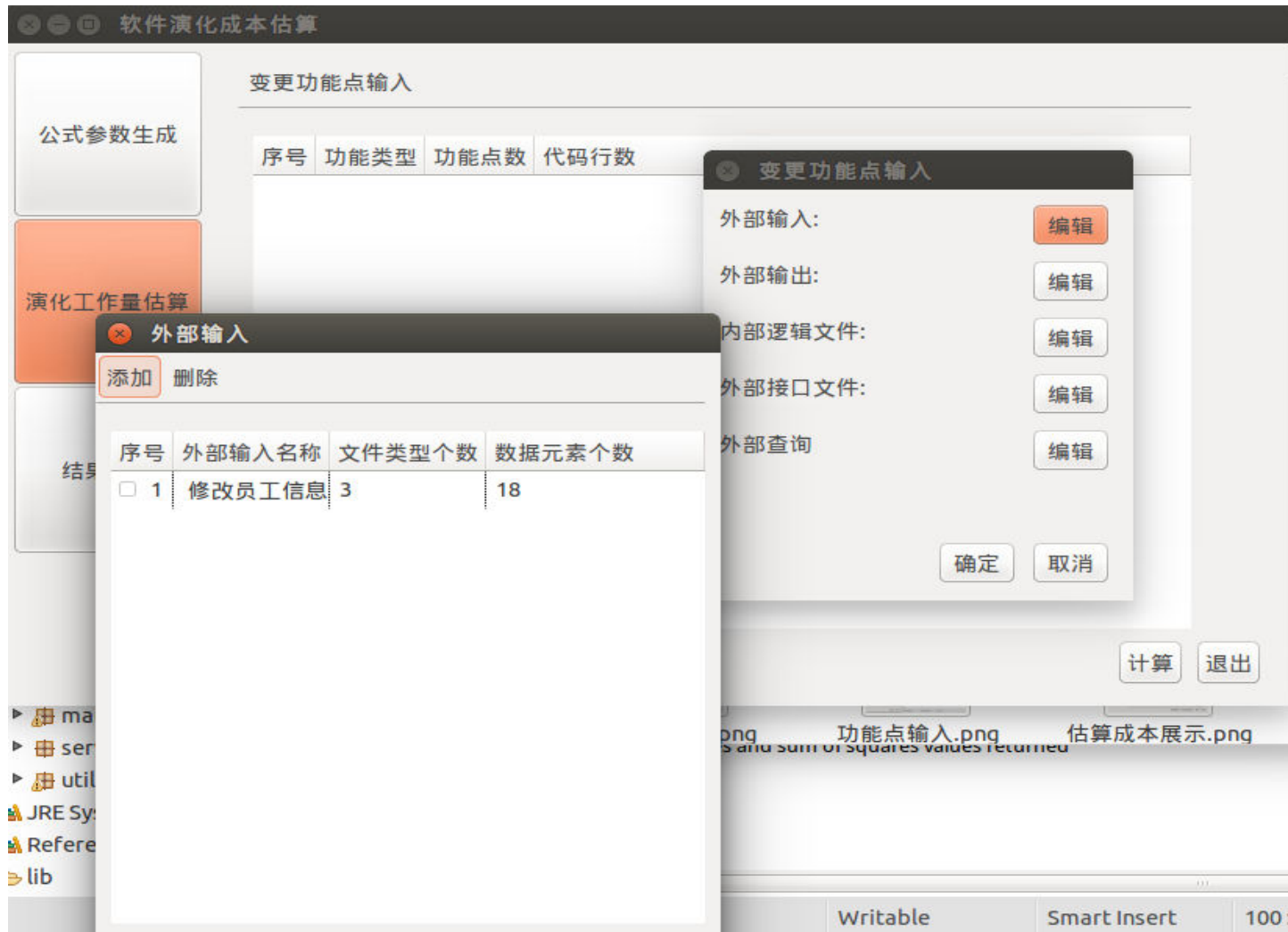
Typical Function-Oriented Metrics

- ❖ errors per FP (thousand lines of code)
- ❖ defects per FP
- ❖ \$ per FP
- ❖ pages of documentation per FP
- ❖ FP per person-month

典型的面向功能的度量

- 每功能点 (FP) 的错误数
- 每功能点 (FP) 的缺陷数
- 每功能点的成本 (\$ per FP)
- 每功能点的文档页数
- 每人月的功能点 (FP per person-month)

Example



Example

软件演化成本估算

公式参数生成

演化工作量估算

结果展示

变更功能点输入

序号	功能类型	功能点数	代码行数
☐ 1	外部输入	15	795
☐ 2	外部输出	4	212
☐ 3	内部逻辑	14	742
☐ 4	外部接口	5	265
☐ 5	外部查询	6	318

预测变更源代码行数: 2332.0SLOC
预测演化成本: 359.69838937940585人日

计算

退出

Comparing LOC and FP

Programming language	LOC per Function point			
	avg.	M	L	H
Access	35	38	15	47
Ada	154	—	104	205
APS	86	83	20	184
ASP	62	—	32	127
Assembler	337	315	91	694
C	162	109	33	704
C++	66	53	29	178
Java	63	53	77	70
COBOL	77	77	14	400
SQL	40	37	7	110
VBScript	34	27	50	—
Visual Basic	47	42	16	158

Why Opt for FP?

- ❖ **Programming language independent**
- ❖ **Used readily countable characteristics that are determined early in the software process**
- ❖ **Does not “penalize（惩罚）” inventive (short) implementations that use fewer LOC than other more clumsy versions**
- ❖ **Makes it easier to measure the impact of reusable components**

- 编程语言无关
- 使用在软件过程早期确定的易于计数的特征
- 不“惩罚”使用更少代码行（LOC）的发明性（短）实现，而其他较笨拙的版本则会被惩罚
- 更容易衡量可重用组件的影响

Object-Oriented Metrics

- ❖ Number of **scenario scripts** (use-cases)
- ❖ Number of **support classes** (required to implement the system but are not immediately related to the problem domain)
- ❖ Average number of **support classes per key class** (analysis class)
- ❖ Number of **subsystems** (an aggregation of classes that support a function that is visible to the end-user of a system)

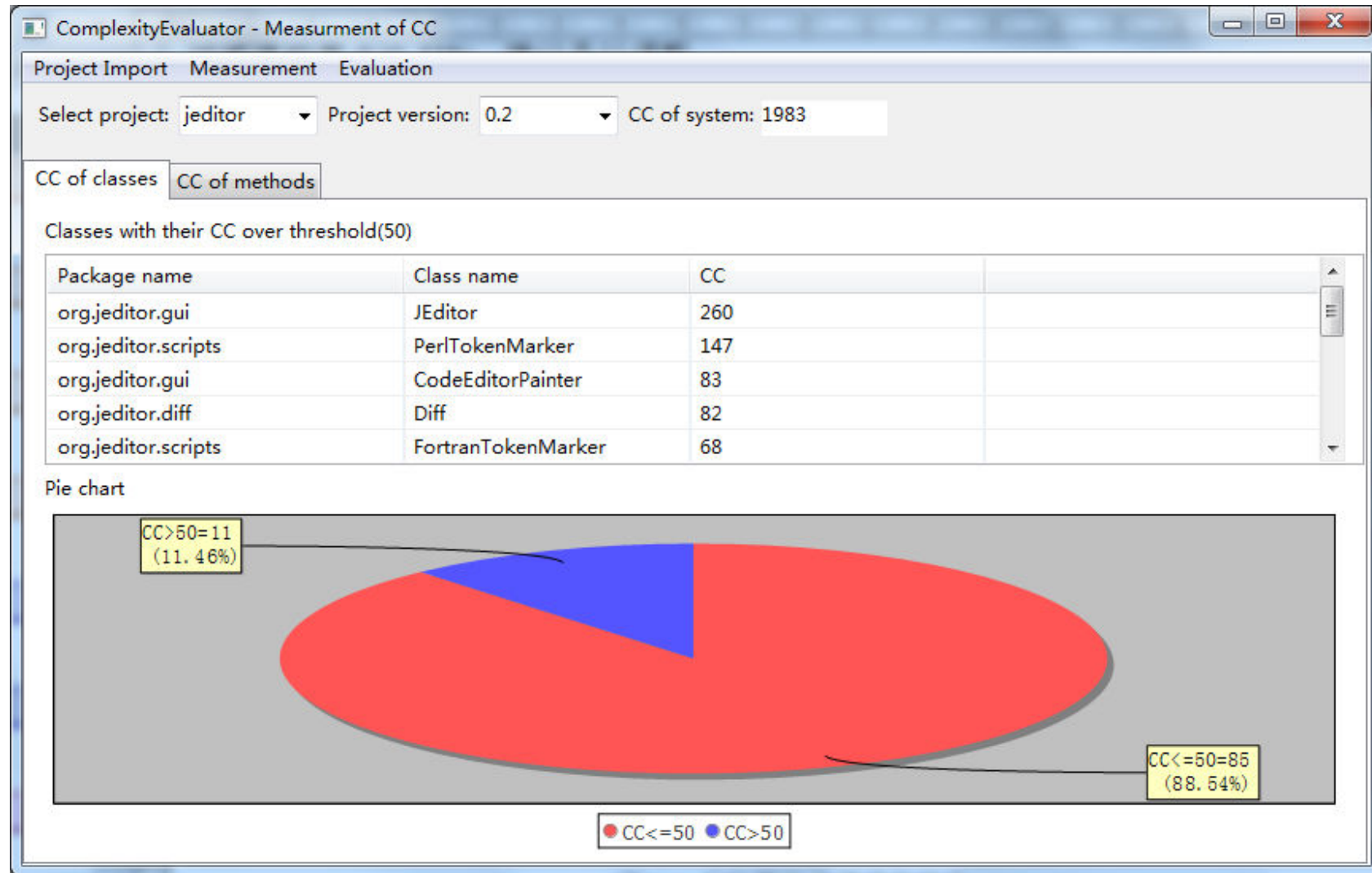
• 使用场景脚本（用例）的数量

• 实现系统所需但与问题域无直接关系的支持类的数量

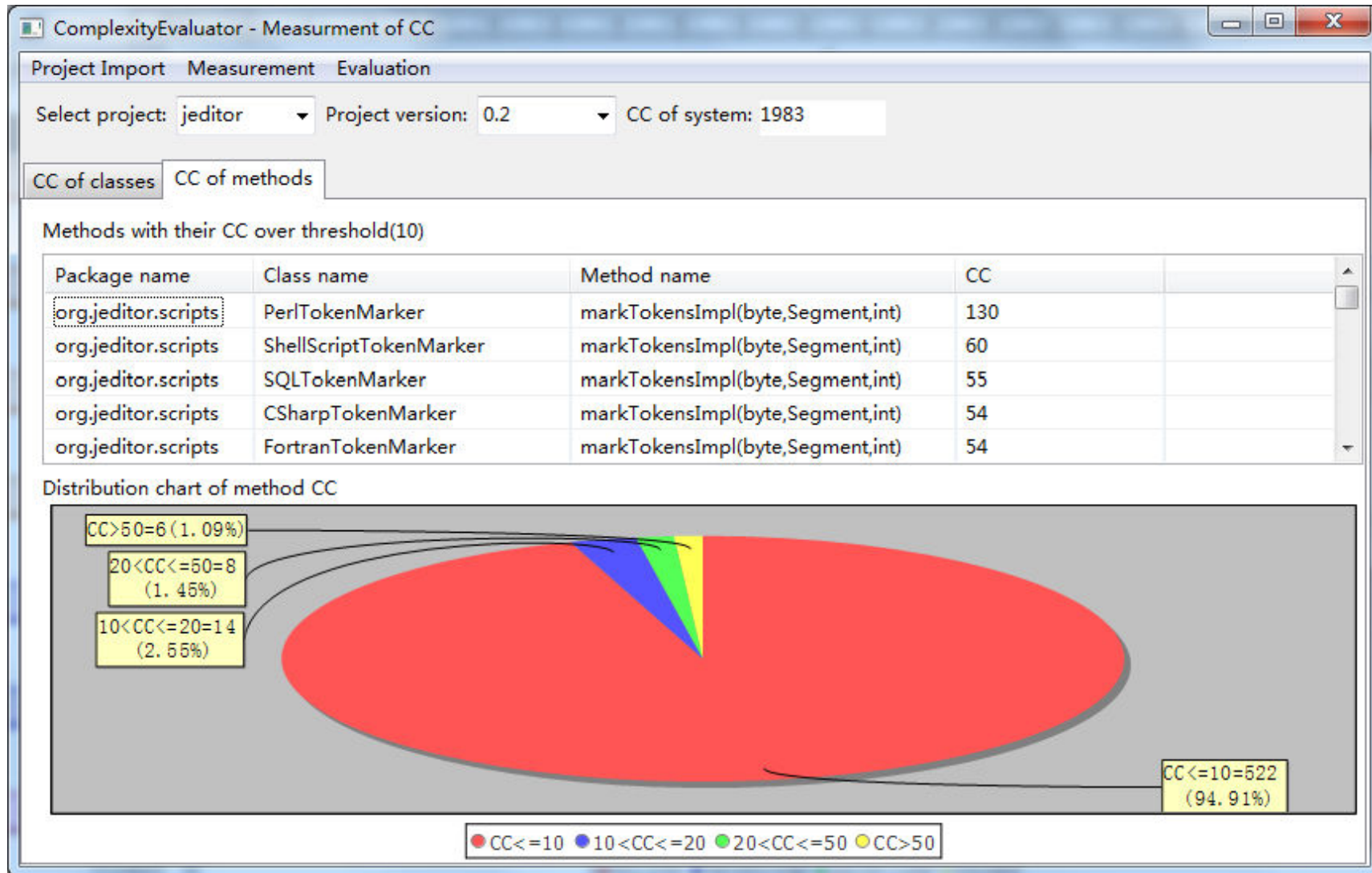
— • 每关键类（分析类）的平均支持类数量

• 支持系统可见功能的子系统的数量

Example



Example



WebApp Project Metrics

- ❖ Number of **static Web pages** (the end-user has no control over the content displayed on the page)
- ❖ Number of **dynamic Web pages** (end-user actions result in customized content displayed on the page)
- ❖ Number of **internal page links** (internal page links are pointers that provide a hyperlink to some other Web page within the WebApp)

❖ Number of **persistent data objects**

❖ Number of **external systems interfaced**

❖ Number of **static content objects**

❖ Number of **dynamic content objects**

❖ Number of **executable functions**

• 持久数据对象的数量

• 接口外部系统的数量

• 静态内容对象的数量

• 动态内容对象的数量

• 可执行功能的数量

• 静态网页的数量（最终用户无法控制页面上显示的内容）

• 动态网页的数量（最终用户操作导致页面上显示自定义内容）

• 内部页面链接的数量（内部页面链接是指向Web应用内其他网页的超链接指针）

Metrics Guidelines

- ❖ Use common sense and organizational sensitivity when interpreting metrics data.
- ❖ Provide regular feedback to the individuals and teams who collect measures and metrics.
- ❖ *Don't use metrics to appraise individuals.*
- ❖ Work with practitioners and teams to set clear goals and metrics that will be used to achieve them.
- ❖ *Never use metrics to threaten individuals or teams.*
- ❖ Metrics data that indicate a problem area should not be considered “negative.” These data are merely an indicator for process improvement.
- ❖ Don't obsess on a single metric to the exclusion of other important metrics.

- 在解释度量数据时使用常识和组织敏感性。
- 向收集度量和度量的个人和团队定期提供反馈。
- 不要使用度量来评估个人。
- 与从业者和团队合作，设定明确的目标和度量标准，以实现这些目标。
- 永远不要使用度量来威胁个人或团队。
- 指示问题区域的度量数据不应被视为“负面”，这些数据只是过程改进的指标。
- 不要痴迷于单一度量，忽视其他重要度量。

Measuring Quality

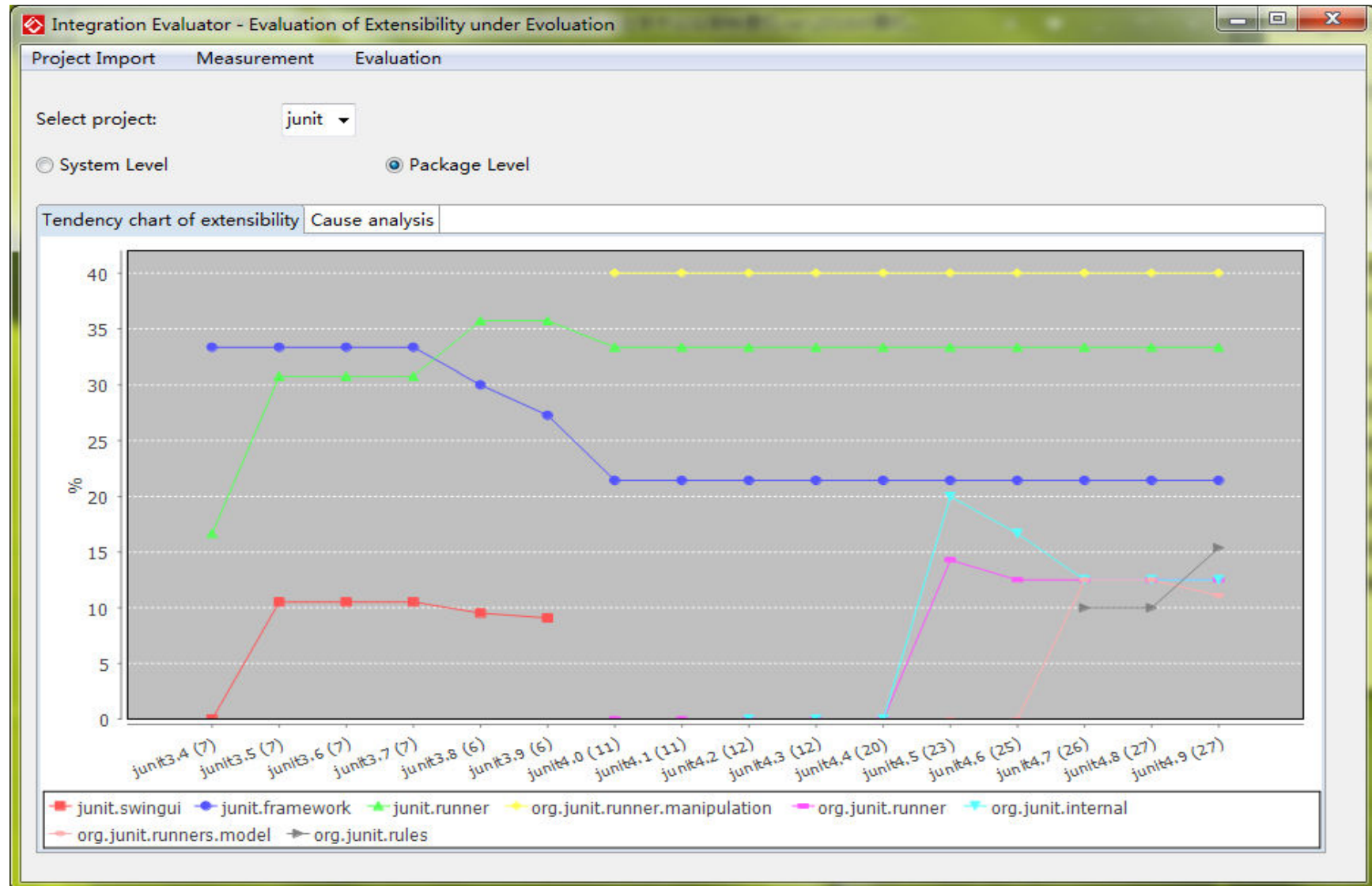
- ❖ **Correctness** — the degree to which a program operates according to specification
- ❖ **Maintainability** — the degree to which a program is amenable to change
- ❖ **Integrity** — the degree to which a program is impervious to outside attack
- ❖ **Usability** — the degree to which a program is easy to use

- **正确性** —— 程序按规范运行的程度
- **完整性** —— 程序不受外部攻击的程度
- **可维护性** —— 程序易于更改的程度
- **可用性** —— 程序易于使用的程度

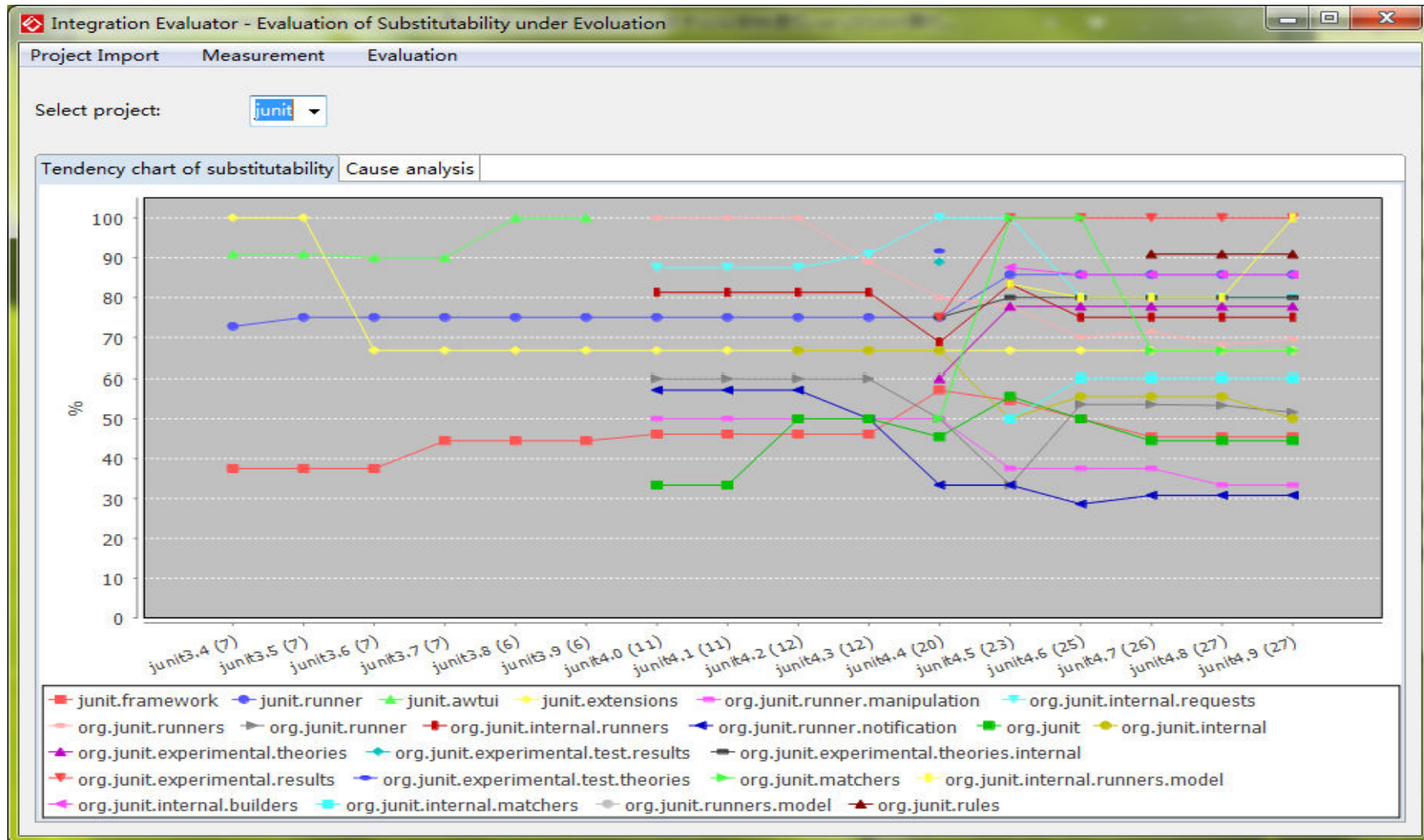
Example



Example



Example





Southeast University



Thank You !

lliao@seu.edu.cn